

# Тема

SQL: Создание, изменение  
и удаление таблиц



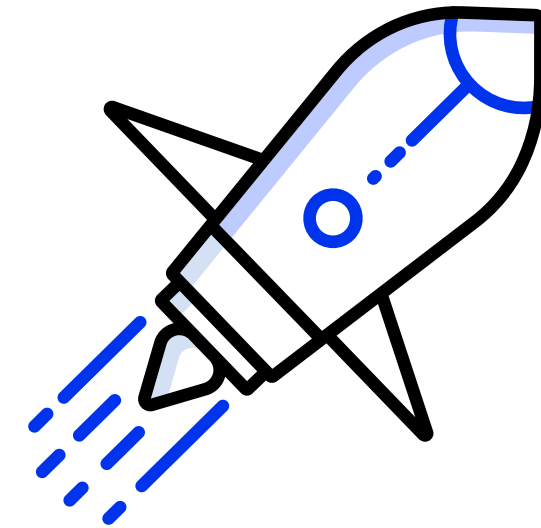
# Анна Морозова

Менеджер-аналитик, Яндекс.Кью

ex-TheQuestion, ex-Ultimate Guitar, ex-  
МегаФон

# Содержание урока

1. Зачем и как менять таблицы?
2. Создание таблиц
  - ❑ Создание новых таблиц
  - ❑ Типы данных
  - ❑ Ограничения
  - ❑ Создание временных таблиц
  - ❑ Создание таблиц на основе существующих
3. Изменение таблиц
4. Удаление таблиц
5. Воркшоп: создаем базу данных интернет магазина



**Зачем и как**  
менять таблицы?

# Зачем?



**Необходимо начать** записывать данные в новую таблицу



**Надо добавить** первичный или внешний ключ, чтобы связать таблицы между собой



**Надо изменить** тип данных, потому что при создании таблицы указали не тот формат



**Таблицы объединились** и от некоторые из них оказались лишними



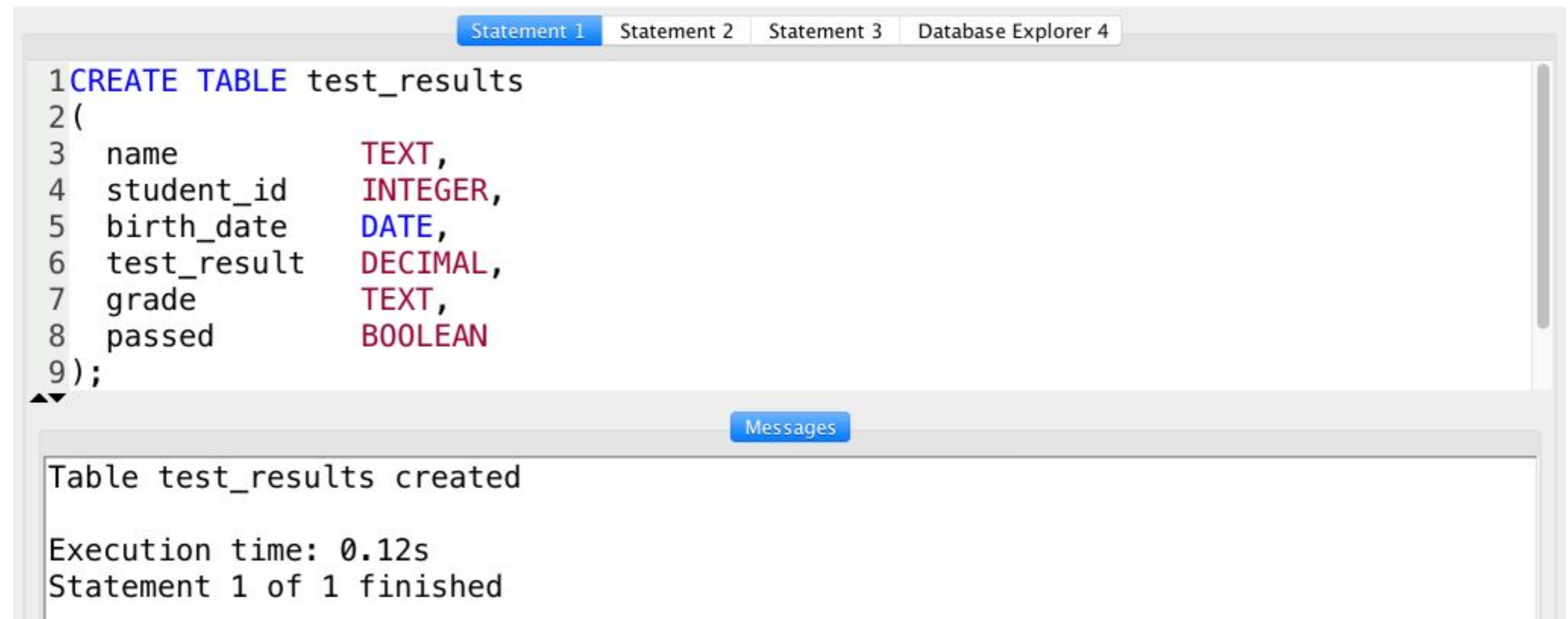
**(Ваш случай)**



# Как это делать?

**#1**

**Вручную  
(SQL запрос)**



The screenshot shows a database management interface with a tabbed window. The active tab is 'Statement 1'. The SQL code in the editor is:

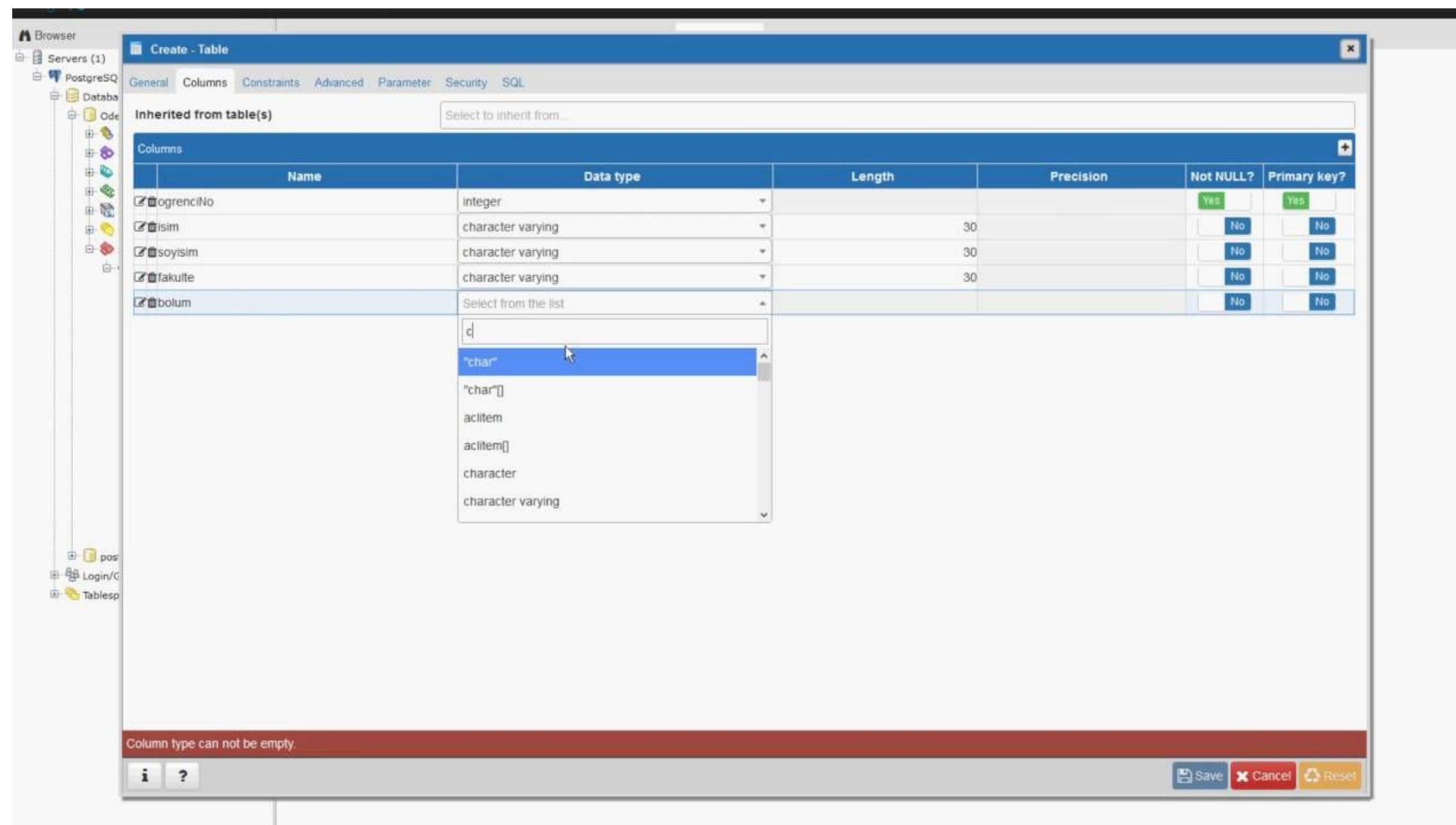
```
1 CREATE TABLE test_results
2 (
3   name          TEXT,
4   student_id    INTEGER,
5   birth_date    DATE,
6   test_result   DECIMAL,
7   grade         TEXT,
8   passed        BOOLEAN
9 );
```

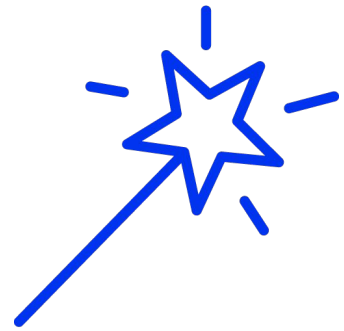
Below the code editor is a 'Messages' pane. It displays the following text:

```
Table test_results created
Execution time: 0.12s
Statement 1 of 1 finished
```

# Как это сделать?

**#2**  
**В СУБД**



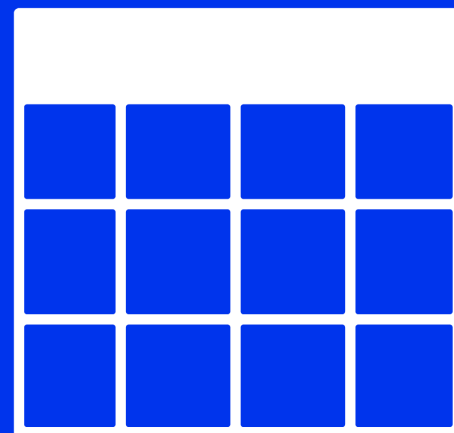


## Совет:

Перед тем как приступить к любому из описанных в лекции действий, откройте и прочитайте документацию к своей СУБД



# Создание таблиц



# **CREATE TABLE**

создает новую таблицу

# Порядок создания таблиц

**#1**

Указать

название таблицы

# Порядок создания таблиц

**#1**

Указать  
название таблицы

**#2**

Указать  
название столбцов

# Порядок создания таблиц

**#1**

Указать  
название таблицы

**#2**

Указать  
название столбцов

**#3**

Указать  
тип данных для столбцов



# Порядок создания таблиц

**#1**

Указать  
название таблицы

**#2**

Указать  
название столбцов

**#3**

Указать  
тип данных для столбцов

**#4**

Указать  
ограничения  
(опционально)

# Порядок создания таблиц

**#1**

Указать  
название таблицы

**#2**

Указать  
название столбцов

**#3**

Указать  
тип данных для столбцов

**#4**

Указать  
ограничения  
(опционально)

# СИНТАКСИС

```
CREATE TABLE table (  
column1 data_type1 [constraints1],  
column2 data_type2 [constraints2],  
...  
columnN data_typeN [constraintsN]  
)
```



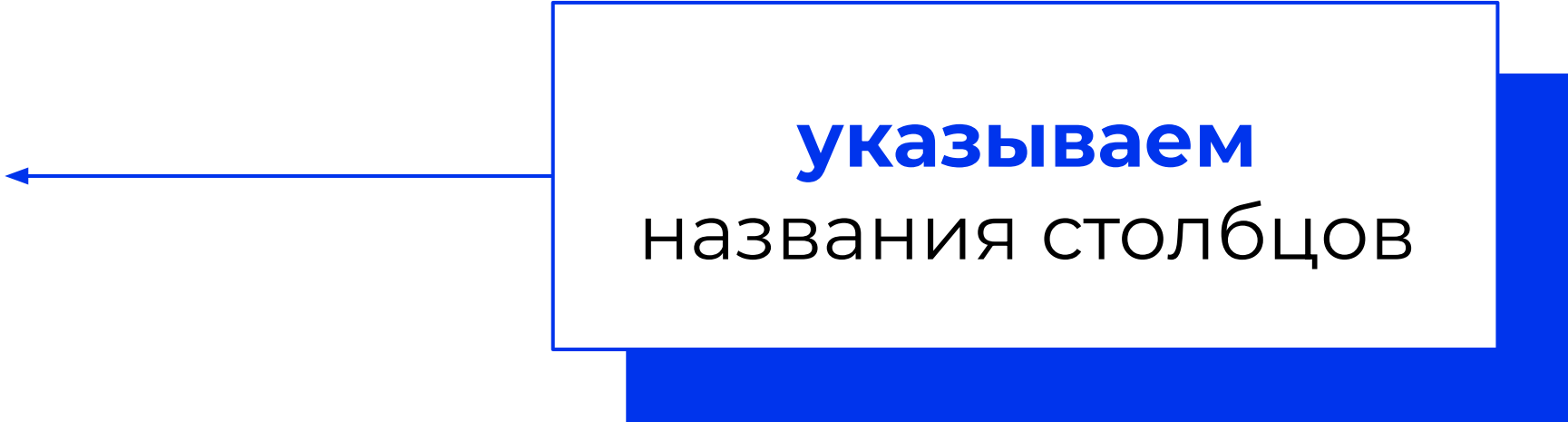
**указываем**  
название таблицы



# СИНТАКСИС

```
CREATE TABLE table (  
column1 data_type1 [constraints1],  
column2 data_type2 [constraints2],  
...  
columnN data_typeN [constraintsN]  
)
```

**указываем**  
названия столбцов

A blue-outlined rectangular callout box with a blue drop shadow is positioned on the right side of the slide. A blue arrow points from the left side of the box to the column names 'column1' and 'column2' in the SQL syntax example. The box contains the text 'указываем' (we indicate) in bold blue font and 'названия столбцов' (column names) in black font.

# СИНТАКСИС

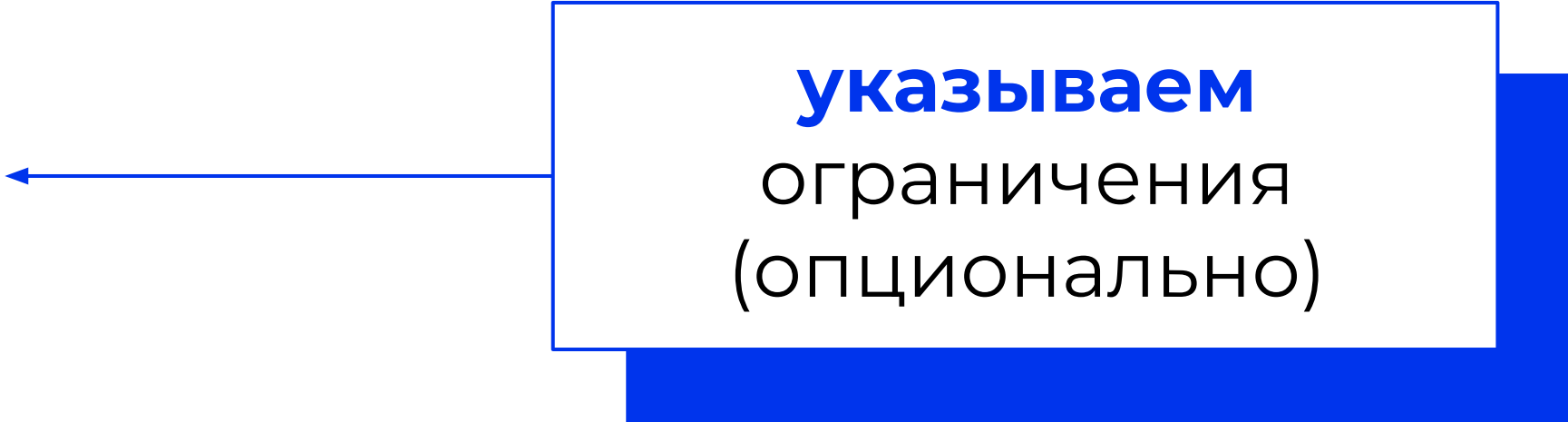
```
CREATE TABLE table (  
column1 data_type1 [constraints1],  
column2 data_type2 [constraints2],  
...  
columnN data_typeN [constraintsN]  
)
```



**указываем**  
ТИПЫ ДАННЫХ ДЛЯ  
СТОЛБЦОВ

# СИНТАКСИС

```
CREATE TABLE table (  
column1 data_type1 [constraints1],  
column2 data_type2 [constraints2],  
...  
columnN data_typeN [constraintsN]  
)
```

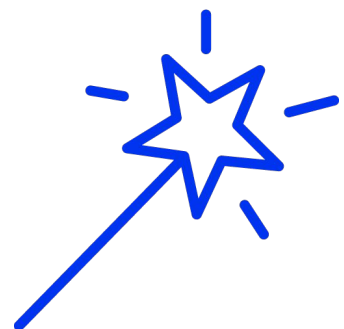


**указываем**  
ограничения  
(опционально)

# Пример

```
CREATE TABLE table (  
column1 data_type1 [constraints1],  
column2 data_type2 [constraints2],  
...  
columnN data_typeN [constraintsN]  
)
```

```
CREATE TABLE books (  
title_id CHAR(3) NOT NULL ,  
title_name VARCHAR(40) NOT NULL ,  
pages INTEGER ,  
price DECIMAL(5,2) ,  
pubdate DATE  
)
```



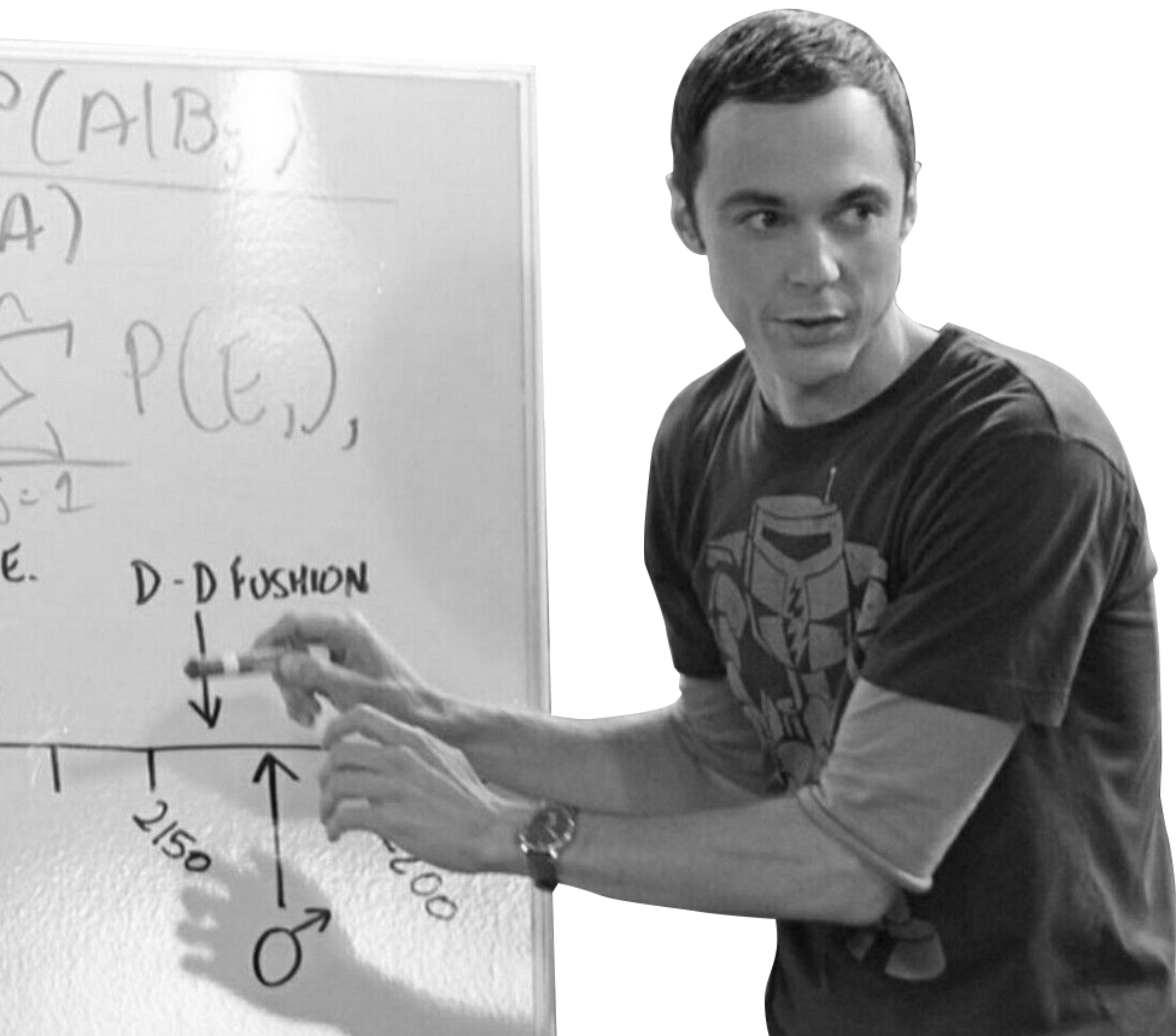
# Советы:

1. Ввод описания каждого столбца таблицы начинается **с новой строки**
2. Названия таблицы в базе данных и каждого столбца в таблице должны быть **уникальными**
3. Сразу после создания таблица пуста (не имеет строк).  
Чтобы **заполнить таблицу данными**, пользуйтесь командой INSERT





# ТИПЫ ДАННЫХ



# Важно!

Всегда проверяйте документацию!  
Типы данных и их свойства  
**отличаются** в разных СУБД

## Текстовые

- ❑ CHAR(n)
  - ❑ VARCHAR(n)
  - ❑ TEXT ...
- 

**Числовые** (целые  
и с плавающей точкой)

- ❑ INT(n)
  - ❑ FLOAT(n)
  - ❑ REAL ...
- 

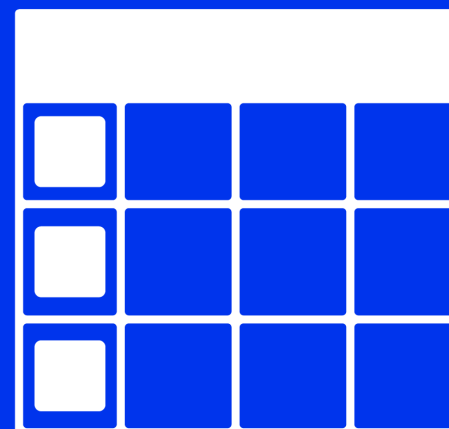
## Дата и время

- ❑ DATE
  - ❑ DATETIME
  - ❑ TIMESTAMP ...
- 

## Другие

- ❑ (очень много)





# ОГРАНИЧЕНИЯ

позволяют указать требования к значениям

# Ограничение **NOT NULL**

Не разрешает присваивать  
столбцу значение null

```
CREATE TABLE users (  
    id INT NOT NULL  
)
```

# Ограничение **DEFAULT**

Задаёт для столбца значение  
по умолчанию

```
CREATE TABLE users (  
  id INT NOT NULL,  
  city VARCHAR(255) DEFAULT 'Dublin'  
)
```

# Ограничение **PRIMARY KEY**

Задаёт столбец (столбцы)  
первичного ключа для  
таблицы

```
CREATE TABLE users (  
    id INT NOT NULL PRIMARY KEY,  
    city VARCHAR(255) DEFAULT 'Dublin'  
)
```

```
CREATE TABLE users (  
    id INT NOT NULL,  
    city VARCHAR(255) DEFAULT 'Dublin',  
    PRIMARY KEY (id)  
)
```

# Ограничение **FOREIGN KEY**

Задаёт столбец (столбцы)  
внешнего ключа для таблицы

```
CREATE TABLE orders (  
    id INT NOT NULL PRIMARY KEY,  
    user_id INT REFERENCES users(id)  
)
```

```
CREATE TABLE orders (  
    id INT NOT NULL,  
    user_id INT,  
    PRIMARY KEY (id),  
    FOREIGN KEY (user_id) REFERENCES  
    users(id)  
)
```



# Ограничение **UNIQUE**

Не разрешает добавлять  
в столбец повторяющиеся  
значения

```
CREATE TABLE users (  
  id INT NOT NULL PRIMARY KEY,  
  city VARCHAR(255) DEFAULT 'Dublin',  
  email VARCHAR(255) NOT NULL UNIQUE  
)
```

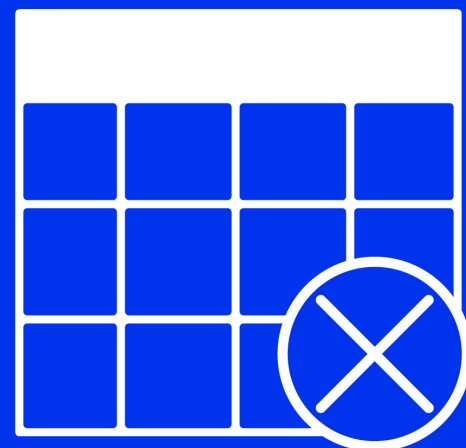
```
CREATE TABLE users (  
  id INT NOT NULL,  
  city VARCHAR(255) DEFAULT 'Dublin',  
  email VARCHAR(255) NOT NULL,  
  PRIMARY KEY (id),  
  UNIQUE (email)  
)
```

# Ограничение **CHECK**

Ограничивает значения,  
которые могут добавляться  
в столбец, с помощью  
логических выражений

```
CREATE TABLE users (  
    id INT NOT NULL PRIMARY KEY,  
    age INT,  
    CHECK (age >= 18)  
)
```

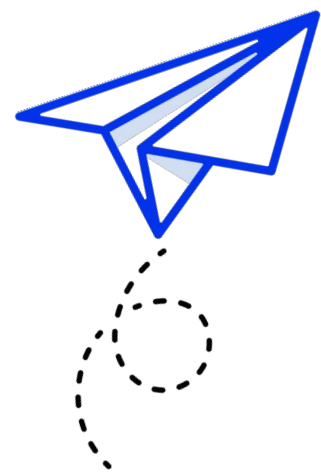
```
CREATE TABLE users (  
    id INT NOT NULL PRIMARY KEY,  
    age INT CHECK (age >= 18)  
)
```



# CREATE TEMPORARY TABLE

создает временную таблицу, которая затем  
будет удалена



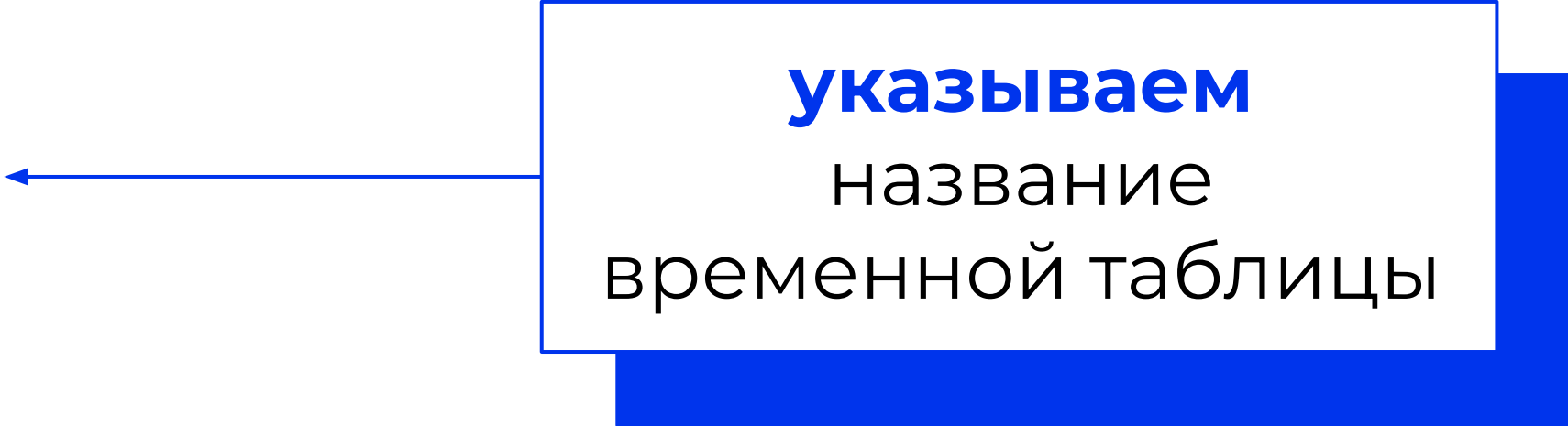


# Принципы работы

Создаются для **временного хранения**  
данных и **удаляются** в конце сессии

# Синтаксис

```
CREATE TEMPORARY TABLE table (  
column1 data_type1 [constraints1],  
column2 data_type2 [constraints2],  
...  
columnN data_typeN [constraintsN]  
)
```



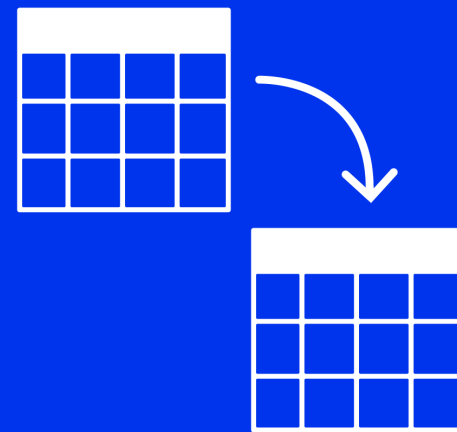
**указываем**  
название  
временной таблицы

The diagram consists of a white rectangular box with a blue border. Inside the box, the text 'указываем' is in bold blue, and 'название временной таблицы' is in black. A blue arrow points from the left side of the box to the word 'table' in the SQL syntax above. To the right of the box is a solid blue L-shaped graphic element.

# Пример

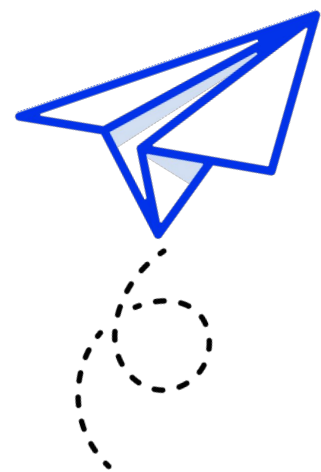
```
CREATE TEMPORARY TABLE table (  
column1 data_type1 [constraints1],  
column2 data_type2 [constraints2],  
...  
columnN data_typeN [constraintsN]  
)
```

```
CREATE TEMPORARY TABLE editors (  
id CHAR(3) NOT NULL PRIMARY KEY,  
first_name VARCHAR(15) ,  
last_name VARCHAR(15) ,  
phone VARCHAR(12) ,  
pub_id CHAR(3)  
)
```



# **SELECT INTO**

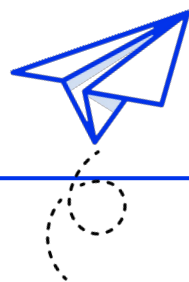
создает новую таблицу на основе  
уже существующей



# Принцип работы

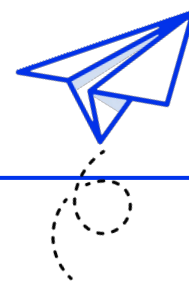
Команда **SELECT INTO** создает **новую таблицу**  
и заполняет ее **результатами исполнения** команды SELECT





## **SELECT INTO**

экспортирует строки  
из существующей таблицы

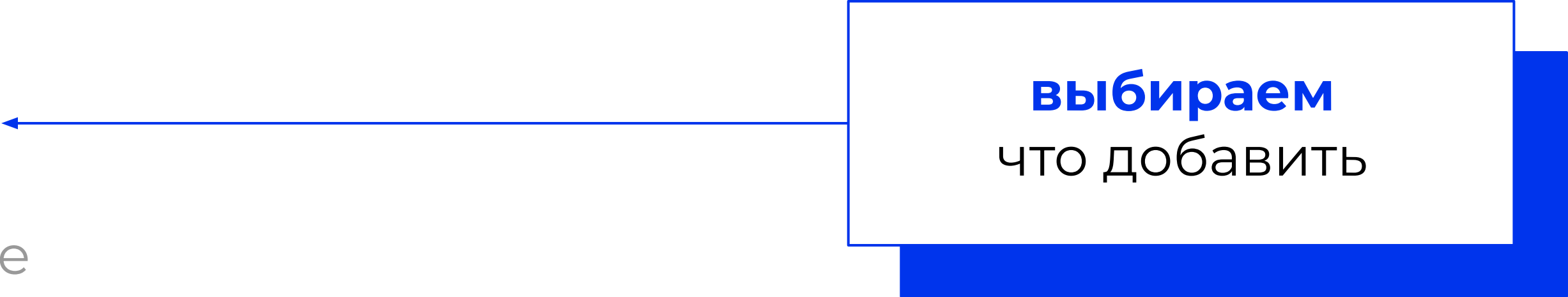


## **INSERT INTO**

импортирует строки  
в существующую таблицу

# СИНТАКСИС

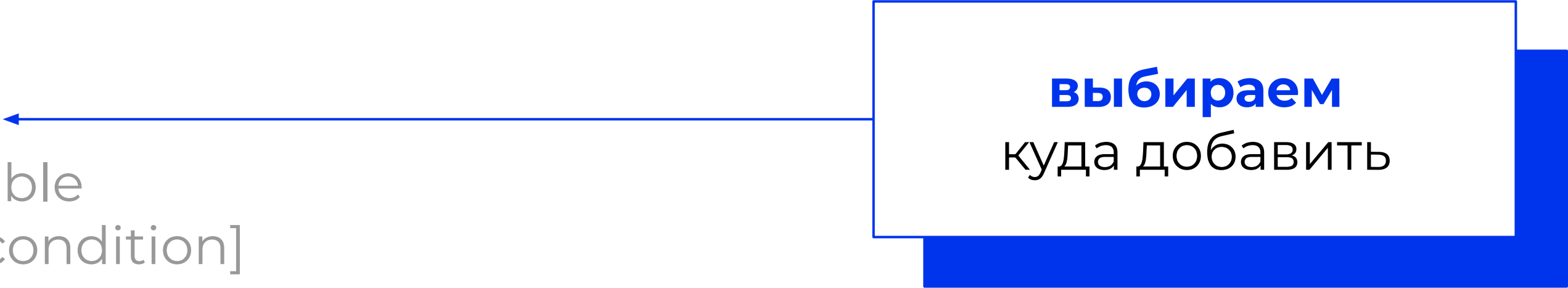
```
SELECT columns  
INTO new_table  
FROM existing_table  
[WHERE search_condition]
```



**выбираем**  
что добавить

# СИНТАКСИС

```
SELECT columns  
INTO new_table  
FROM existing_table  
[WHERE search_condition]
```



**выбираем**  
куда добавить



# СИНТАКСИС

```
SELECT columns  
INTO new_table  
FROM existing_table  
[WHERE search_condition]
```

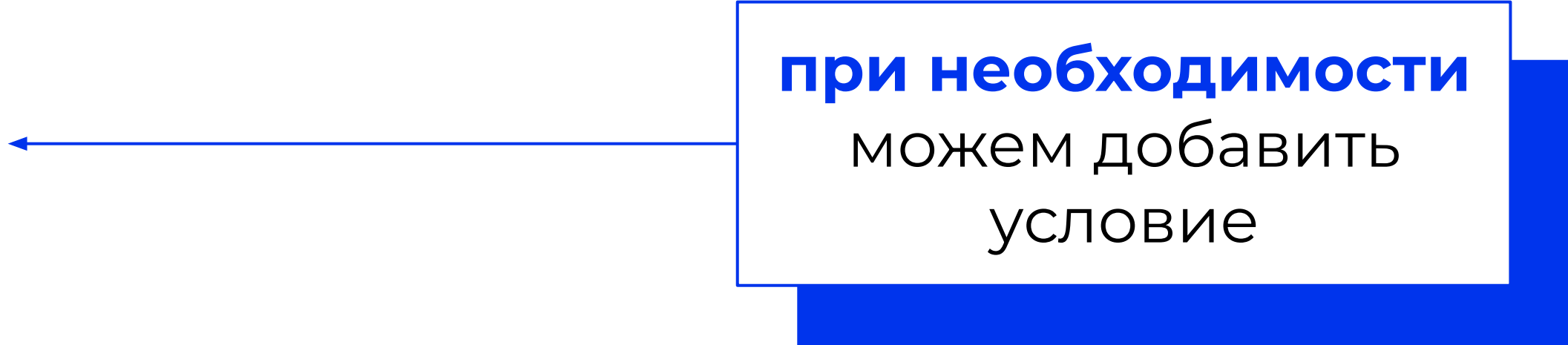


**выбираем**  
откуда взять данные

# СИНТАКСИС

```
SELECT columns  
INTO new_table  
FROM existing_table  
[WHERE search_condition]
```

**при необходимости**  
можем добавить  
условие

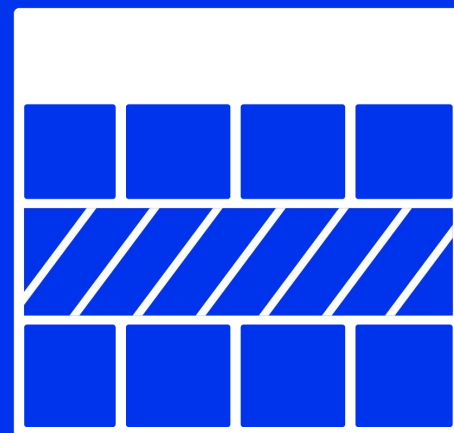
A blue rectangular callout box with a thin blue border and a thick blue L-shaped shadow on its right and bottom edges. A horizontal blue arrow points from the left side of the box to the blue 'WHERE' keyword in the SQL syntax above.

# Пример

```
SELECT columns  
INTO new_table  
FROM existing_table  
[WHERE search_condition]
```

```
SELECT *  
INTO authors2  
FROM authors
```

**Изменение таблиц**



# **ALTER TABLE**

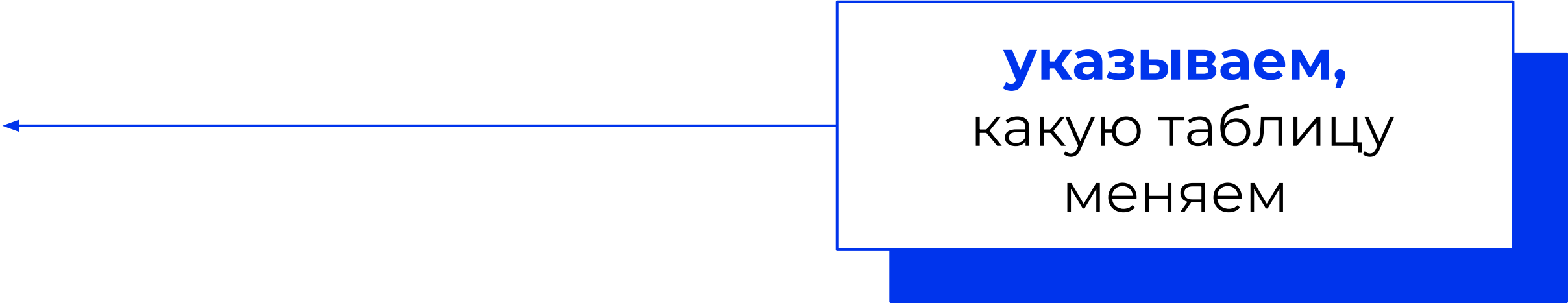
изменяет структуру существующей таблицы



# Синтаксис

`ALTER TABLE table`  
`alter_table_action`

**указываем,**  
какую таблицу  
меняем



# Синтаксис

ALTER TABLE table  
alter\_table\_action

**указываем,**  
тип действия  
и объект

**действие:**  
ADD (добавить)  
ALTER (изменить)  
DROP (удалить)

**объект:**  
Столбец  
Ключ  
Ограничение

# Добавить столбец

## ADD COLUMN

```
ALTER TABLE table  
ADD COLUMN column1 data_type1  
col_constraints1
```

```
ALTER TABLE users  
ADD COLUMN email VARCHAR(255)  
NOT NULL
```



# Добавить столбец

## ADD COLUMN

```
ALTER TABLE table  
ADD COLUMN column1 data_type1  
col_constraints1,  
column2 data_type2 col_constraints2
```

```
ALTER TABLE users  
ADD COLUMN email VARCHAR(255)  
NOT NULL,  
birthdate date NULL
```

# Удалить столбец

## **DROP COLUMN**

```
ALTER TABLE table  
DROP COLUMN column1
```

```
ALTER TABLE users  
DROP COLUMN birthdate
```

# Изменить столбец

## **ALTER COLUMN**

```
ALTER TABLE table  
ALTER COLUMN column1 data_type1  
constraints1
```

```
ALTER TABLE users  
ALTER COLUMN birthdate year NOT  
NULL
```

**Добавить  
первичный ключ  
ADD PRIMARY KEY**

```
ALTER TABLE table  
ADD PRIMARY KEY (column)
```

```
ALTER TABLE users  
ADD PRIMARY KEY (id)
```

# Добавить внешний ключ **ADD FOREIGN KEY**

```
ALTER TABLE table1  
ADD FOREIGN KEY (column1)  
REFERENCES table2(column2)
```

```
ALTER TABLE orders  
ADD FOREIGN KEY (user_id)  
REFERENCES users(id)
```



# Удалить первичный ключ **DROP PRIMARY KEY**

```
ALTER TABLE table  
DROP PRIMARY KEY
```

```
ALTER TABLE users  
DROP PRIMARY KEY
```

# Удалить внешний ключ **DROP FOREIGN KEY**

```
ALTER TABLE table  
DROP FOREIGN KEY FK_column
```

```
ALTER TABLE orders  
DROP FOREIGN KEY FK_user_id
```

**Удаление таблиц**





# **DROP TABLE**

удаляет таблицу и все ее данные

# **DROP TABLE**

удаляет и записи,  
и саму таблицу

# **DELETE FROM**

удаляет только записи,  
но оставляет таблицу

# Синтаксис

`DROP TABLE table`

букв мало, а проблем после много



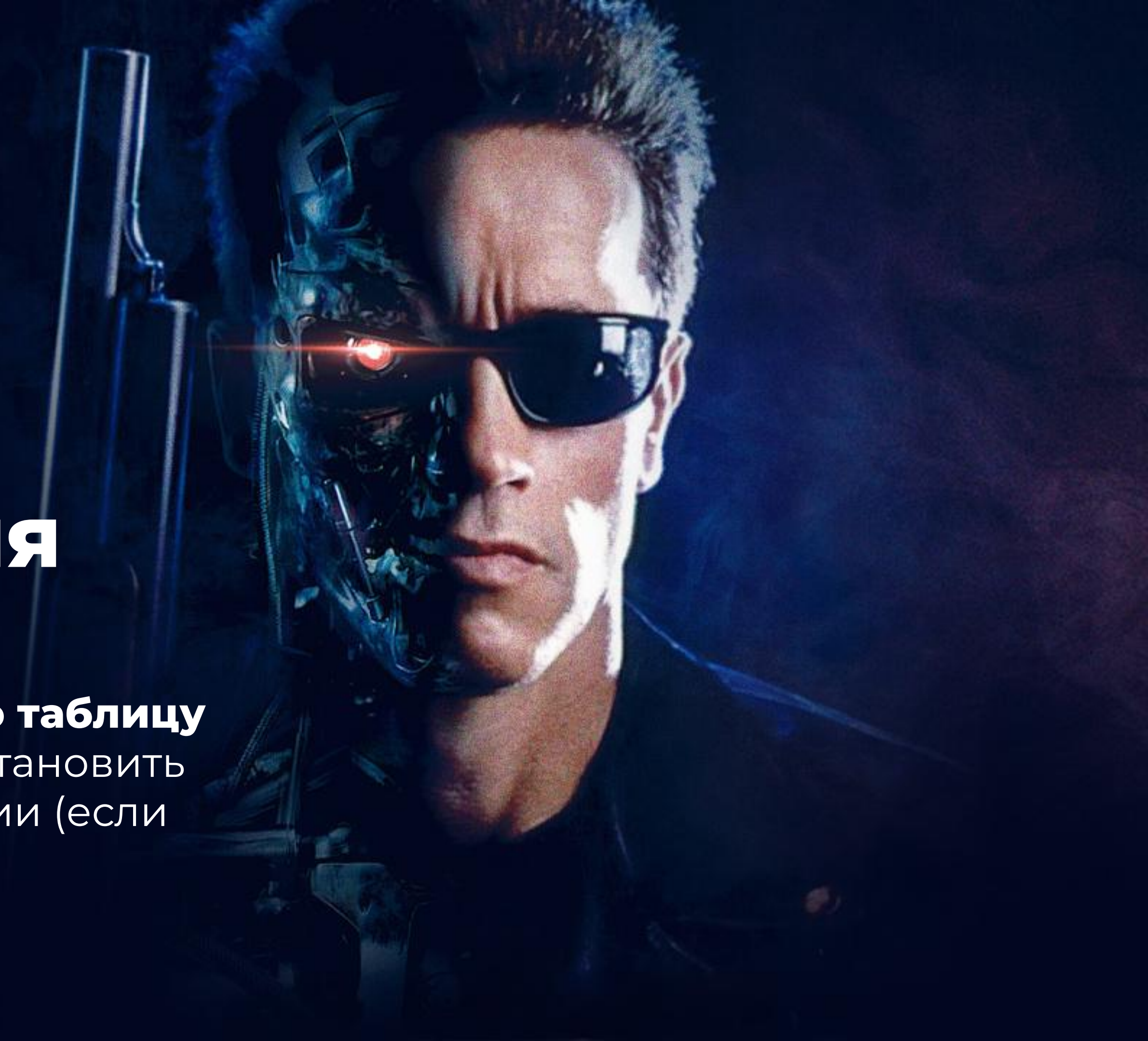
# Последствия

- ❑ Команду DROP TABLE **нельзя отменить**, таблица не подлежит восстановлению;
- ❑ Будет **удалено всё**: структура таблицы, данные, индексы, ограничения, права доступа и т.д.;
- ❑ Появятся **сложности** с внешними ключами (если их не изменить и не удалить).

# Последствия

**Единственный способ  
восстановить удаленную таблицу**

– заново создать ее и восстановить  
данные из резервной копии (если  
она была)





# Воркшоп

## Создаем базу данных интернет магазина

